

Lab09-VirtualMemory

题目:

在Linux（或 Windows）平台上，采用C语言编写一个程序来模拟请求调页存储管理的过程—实现对页面、页表、地址转换和页面置换过程等，以此加深对请求调页系统的原理和实现过程的理解。

要求:

- 假设每个页面中可存放10条指令，分配给一个作业的内存块数为4；
- 一作业，共有320条指令，即它的地址空间为32页，目前它的所有页都还未调入内存。
 - a. 在模拟过程中，如果所访问的指令已在内存，则显示其物理地址，并转下一条指令；
 - b. 如果所访问的指令还未装入内存，则发生缺页，此时需记录缺页的次数，并将相应页调入内存；
 - c. 如果4个内存块均已装入该作业，则需进行**页面置换**；
 - d. 最后显示其物理地址，并转入下一条指令；
 - e. 在所有 320 条指令执行完毕后，请计算并显示作业运行过程中发生的缺页率；
- 置换算法：请分别考虑**OPT**、**FIFO**和**LRU**算法，并给出命中率；其中命中率为（本例中页地址流长度为 320）：

$$\text{命中率} = 1 - \text{页面失效次数} / \text{页地址流长度}$$

作业中指令的访问次序按下述原则生成：

1. 50%的指令是顺序执行的；
2. 25%的指令是均匀分布在前地址部分；
3. 25%的指令是均匀分布在后地址部分；
4. 具体的实施办法是：
 - a. 在[0,319]之间随机选取一条起始执行指令，其序号为m；
 - b. 顺序执行下一条指令，即序号为 m+1 的指令；
 - c. 通过随机数，跳转到前地址部分[0,m-1]中的某条指令处，其序号为 m_1 ；
 - d. 顺序执行下一条指令，即序号 m_1+1 的指令；
 - e. 通过随机数，跳转到后地址部分[$m_1+2,319$]中的某条指令处，其序号为 m_2 ；
 - f. 顺序执行下一条指令，即序号 m_2+1 的指令；

- g. 重复跳转到前地址部分、顺序执行、跳转到后地址部分、顺序执行的过程，直至执行 320 指令。

思考题：

1. 如果增加分配给作业的内存块数，将会对作业运行过程中的缺页率产生什么影响；
2. 为什么一般情况下，LRU 具有比 FIFO 更好的性能？

提交成果：

实验报告，包含以下内容：

源程序

具体实例结果文件

思考题的讨论

提交格式：

上述源文件格式：os_lab_vmpage_<学号>_<姓名>

上述测试实例文件：os_lab_vmpage_<学号>_<姓名>_result

上述思考题的讨论：os_lab_vmpage_<学号>_<姓名>_discuss

上述文件统一压缩打包为：

os_lab_vmpage_<学号>_<姓名>.tgz

or os_lab_vmpage_<学号>_<姓名>.doc

相关头文件：

#include <stdio.h>

#include <string.h>

#include <unistd.h>

linux 相关参考函数：

fork、execvp、wait：创建进程

strcmp、strcpy、strncpy：字符串相关操作

fopen、fclose、fscanf、fprintf、fgets、fputs：文件 stdio 操作

sprintf：任意类型转换为字符串

atoi：字符串转换为整数类型 int

random：随机函数